CS2008 Numerical Computing

Fall 2025

Assignment 1

Due: 14 Oct 2025 (in Google Classroom)

Instructions

1. **Plagiarized work will result in zero marks.**
2. **No retake or late submission will be accepted.**
3. The submission should be a **SINGLE UNZIPPED NOTEBOOK** submitted on **Google Classroom**.
4. This notebook should be **properly documented**: what you did, how you did it, and the source code for each part along with the generated outputs.
5. Your submission file should follow the **format: id_section_A1** e.g., **i22123456_A_A1**. (Note: A1 denotes Assignment 1.)

Learning Objectives

By completing this assignment, you will:

- Apply the Intermediate Value Theorem (IVT) and bracketing strategies.
- Implement classic root-finding methods: **Bisection**, **Fixed-Point**, **Newton–Raphson**, **Secant**, and a **hybrid safeguarded** method.
- Analyze convergence behaviour (order, error curves) and robustness across functions, including multiple roots and flat derivatives.
- Compare accuracy vs. iterations and accuracy vs. time.

Functions Under Test

For each task, solve $f(x) = 0$ on the specified interval(s):

1. $f_1(x) = x^3 + x - 3$, interval $[0, 4]$ (single root).
2. $f_2(x) = \cos x - x$, interval $[0, 1]$.
3. $f_3(x) = xe^{-x} - 0.1$, interval $[0, 2]$.
4. $f_4(x) = x^3 - 2x^2 - 5$, interval $[2, 4]$.
5. $f_5(x) = (x - 1)^2(x + 2)$ (multiple root at $x = 1$), intervals $[-3, 0]$ and $[0, 3]$.
6. $f_6(x) = \sqrt[3]{x}$ (derivative blows up near 0); test near the root at $x = 0$ using small brackets like $[-10^{-3}, 10^{-3}]$.
7. **Your choice** f_7: a nontrivial continuous function with at least one real root in a bracket you provide (state why your bracket is valid).

Tolerances Absolute function tolerance $\varepsilon_f = 10^{-8}$, absolute step tolerance $\varepsilon_x = 10^{-10}$, and *max iterations* = 200 unless otherwise stated.

Q1. IVT & Bracketing Warm-Up (10 marks)

(a) For f_1–f_5, verify the IVT condition on the given interval: continuity + sign change $f(a) \cdot f(b) < 0$. Show $f(a), f(b)$.

(b) For f_6 and your f_7, **construct** a small bracket $[a, b]$ with $f(a)f(b) < 0$ and justify (numerically) why the root lies inside.

Q2. Implement *Bisection* (15 marks)

Implement `bisection(f, a, b, tol_f, tol_x, maxit)` that:

- checks IVT precondition and errors if violated,
- stops when $|f(m)| \leq \varepsilon_f$ *or* $\frac{b-a}{2} \leq \varepsilon_x$,
- returns `root, iters, history` storing $(x_k, f(x_k)$, interval width$)$.

Starter:

```
def bisection(f, a, b, tol_f=1e-8, tol_x=1e-10, maxit=200):
    fa, fb = f(a), f(b)
    if fa == 0.0: return a, 0, [(a, fa, b-a)]
    if fb == 0.0: return b, 0, [(b, fb, b-a)]
    if fa * fb > 0:
        raise ValueError("Bisection precondition violated: f(a)*f(b) > 0")

    history = []
    for k in range(1, maxit+1):
        m = 0.5*(a+b)
        fm = f(m)
        history.append((m, fm, b-a))
        if abs(fm) <= tol_f or 0.5*(b-a) <= tol_x:
            return m, k, history
        if fa * fm < 0:
            b, fb = m, fm
        else:
            a, fa = m, fm
    return m, k, history
```

Q3. Implement *Fixed-Point Iteration* (10 marks)

Choose $g(x)$ (at least for f_1, f_2, f_3) so that $x = g(x)$ has the same root. Discuss $|g'(x)|<1$. Implement:

```
def fixed_point(g, x0, tol_x=1e-10, maxit=200):
    x = x0
    hist = [x]
    for k in range(maxit):
        xn = g(x)
        hist.append(xn)
        if abs(xn - x) <= tol_x:
            return xn, k+1, hist
        x = xn
    return x, maxit, hist
```

Experiment with two g's per function; report successes/failures.

Q4. *Newton–Raphson* and *Secant* (20 marks)

Newton with analytic derivative (or safe numerical); Secant derivative-free.

```
def newton(f, df, x0, tol_f=1e-8, tol_x=1e-10, maxit=100, tiny=1e-14):
    x = x0
    hist = []
    for k in range(1, maxit+1):
        fx = f(x)
        dfx = df(x)
        if abs(dfx) < tiny:
            raise ZeroDivisionError("Zero/near-zero derivative encountered")
        xn = x - fx/dfx
        hist.append((x, fx))
        if abs(fx) <= tol_f or abs(xn - x) <= tol_x:
            return xn, k, hist
        x = xn
    return x, k, hist

def secant(f, x0, x1, tol_f=1e-8, tol_x=1e-10, maxit=100, tiny=1e-14):
    f0, f1 = f(x0), f(x1)
    hist = [(x0, f0)]
    for k in range(1, maxit+1):
        if abs(f1 - f0) < tiny:
            raise ZeroDivisionError("Secant breakdown: f1 ~ f0")
        x2 = x1 - f1*(x1 - x0)/(f1 - f0)
        f2 = f(x2)
        hist.append((x1, f1))
        if abs(f2) <= tol_f or abs(x2 - x1) <= tol_x:
            return x2, k, hist
        x0, f0, x1, f1 = x1, f1, x2, f2
    return x1, k, hist
```

Try 2–3 initial guesses (or pairs) per f_i and note failures/successes.

Q5. *Hybrid Safeguarded* (Bisection + Newton) (15 marks)

Maintain a valid bracket $[a, b]$; attempt a Newton step from the midpoint (or best point). If the step leaves the bracket or fails to reduce $|f|$, fall back to bisection. Return root, iterations, and a step-type log. Test on f_1–f_5.

Q6. Convergence and Error Analysis (15 marks)

For each method (Bisection, Fixed-Point (best g), Newton, Secant, Hybrid) and at least f_1, f_2, f_5:

1. Plot **semilogy** of $|f(x_k)|$ vs. iteration k.
2. Plot **semilogy** of $|x_k - x^|$ vs. k (use your best x).
3. Estimate the observed order of convergence p.
4. Discuss linear/superlinear/quadratic behaviour and the multiple root effect for f_5.

Q7. Accuracy vs. Time (10 marks)

Time to reach $|f(x)| \leq 10^{-10}$ on f_1, f_2, f_3 for each method. Provide a table and a grouped bar chart; discuss robustness vs. speed.

Q8. Reflection & Robustness (5 marks)

- Which method would you ship as a default? Why?
- When avoid Newton? When avoid Fixed-Point?
- How do tolerances risk misidentifying a root near a flat spot?

Marks Breakdown (100)

Q1 IVT/Bracketing	10
Q2 Bisection (correctness + design)	15
Q3 Fixed-Point (design + discussion)	10
Q4 Newton & Secant (correctness + tests)	20
Q5 Hybrid (robustness + log)	15
Q6 Convergence & Error plots + discussion	15
Q7 Timing study + plot	10
Q8 Reflection	5
Total	**100**

Notebook Checklist

- Reusable functions with docstrings for each method.
- Helpers for histories and `matplotlib` plots.
- Tables: roots, iterations, final $|f(x)|$, timing.
- Notes on failures (if any) and how the hybrid handled them.

Minimal Driver Skeleton

```
import math, time
import numpy as np
import matplotlib.pyplot as plt

def f1(x): return x**3 + x - 3
def df1(x): return 3*x**2 + 1

def f2(x): return math.cos(x) - x
def df2(x): return -math.sin(x) - 1

def f3(x): return x*math.exp(-x) - 0.1
def df3(x): return math.exp(-x) * (1 - x)

def f4(x): return x**3 - 2*x**2 - 5
def df4(x): return 3*x**2 - 4*x
```

```
def f5(x): return (x-1)**2 * (x+2)
def df5(x): return 2*(x-1)*(x+2) + (x-1)**2

def f6(x): return np.cbrt(x)            # real cube root
def df6(x):                               # undefined at x=0; use carefully
    return (1.0/3.0) * (np.abs(x)**(-2.0/3.0))
# TODO: implement: bisection, fixed_point, newton, secant, hybrid
# TODO: plotting helpers and experiments
```

Hint: Use IVT and sign-change checks in bisection; in the hybrid, "Newton if safe, else bisect."

Academic integrity: This is an individual assignment. Cite any external references you consult.